

TuniQuest: Android Quiz Application

Technical Report

Mohamed Belgacem

Android Development

May 6, 2026

Abstract

This technical report details the design, implementation, and architecture of TuniQuest, a production-ready Android application built with Kotlin and Jetpack Compose. The application delivers an interactive quiz experience focused on Tunisian heritage, employing a clean architecture pattern with separation of concerns between UI, data, and presentation layers. This report covers the project structure, key technologies, data model design, and the implementation of the question loading and repository system using JSON assets and Gson serialization.

Keywords: Android, Kotlin, Jetpack Compose, MVVM, Clean Architecture, Gson, JSON

Contents

1 Introduction

TuniQuest is a Kotlin-based Android application designed to deliver engaging educational content about Tunisian heritage through an interactive quiz format. The application implements industry best practices in Android development, leveraging modern frameworks such as Jetpack Compose for UI design and following MVVM (Model-View-ViewModel) architectural patterns.

1.1 Objectives

1. Provide a robust, maintainable codebase with clear separation of concerns
2. Implement a production-ready data layer for question management
3. Create a responsive, modern user interface using Jetpack Compose
4. Establish a solid foundation for future feature expansion
5. Ensure code quality and testability throughout the application

1.2 Scope

This report focuses on the architectural design and implementation of the core data layer, question loading mechanism, and repository pattern. The UI implementation and ViewModel wiring are beyond the scope of this initial release but are designed with extensibility in mind.

2 Architecture & System Design

2.1 Architectural Pattern

TuniQuest employs a **Clean Architecture** approach with layered separation:

- **Presentation Layer:** Jetpack Compose screens, Navigation, and UI state management
- **ViewModel Layer:** StateFlow-based state holders for business logic coordination
- **Data Layer:** Repository pattern, data models, and data sources (loaders)
- **Model Layer:** Domain entities representing core concepts (Question, Category, Difficulty)

2.2 Project Structure

```
1 com/example/mohamedbelgacem/  
2     data/  
3         model/  
4             Question.kt  
5             Category.kt  
6             Difficulty.kt
```

```
7         loader/  
8             QuestionLoader.kt  
9         repository/  
10             QuestionRepository.kt  
11     ui/  
12         screens/  
13             SplashScreen.kt  
14             MainMenuScreen.kt  
15             CategoryScreen.kt  
16             QuizScreen.kt  
17             SettingsScreen.kt  
18             ResultScreen.kt  
19         theme/  
20             Color.kt  
21             Theme.kt  
22             Type.kt  
23         components/  
24     viewmodel/  
25         QuizViewModel.kt  
26     navigation/  
27         NavRoutes.kt  
28         AppNavigation.kt  
29     MainActivity.kt
```

Listing 1: Project directory layout

3 Data Model Design

3.1 Question Entity

The `Question` data class represents a single quiz question with the following structure:

```
1 data class Question(  
2     val id: Int,  
3     val image: String,  
4     val question: String,  
5     val options: List<String>,  
6     val correctAnswer: Int,  
7     val category: Category,  
8     val difficulty: Difficulty  
9 )
```

Listing 2: Question data class definition

3.1.1 Field Descriptions

Table 1: Question Entity Fields

Field	Type	Description
id	Int	Unique identifier for the question
image	String	Drawable resource name (e.g., “eljem”)
question	String	The quiz question text
options	List<String>	List of answer options
correctAnswer	Int	Index of correct answer (0-based)
category	Category	Enum classifying heritage era
difficulty	Difficulty	Enum indicating question difficulty

3.2 Enumerations

3.2.1 Category Enum

The `Category` enum classifies questions by historical period or theme:

```

1 enum class Category {
2     ROMAN,          // Roman period heritage
3     ISLAMIC,        // Islamic heritage
4     PUNIC,          // Punic/Phoenician period
5     MODERN,         // Modern Tunisian heritage
6     NATURAL,        // Natural heritage and geography
7     CITIES          // Major Tunisian cities
8 }

```

Listing 3: Category enumeration

3.2.2 Difficulty Enum

The `Difficulty` enum represents three levels of question complexity:

```

1 enum class Difficulty {
2     EASY,           // Basic knowledge level
3     MEDIUM,        // Intermediate understanding
4     HARD            // Advanced/expert level
5 }

```

Listing 4: Difficulty enumeration

4 Data Layer Implementation

4.1 Question Loader

The `QuestionLoader` class is responsible for reading and parsing question data from a JSON asset file:

```
1 class QuestionLoader(private val context: Context) {
2
3     fun loadQuestions(): List<Question> {
4         return try {
5             val inputStream = context.assets.open("roman_questions
6                 ↪ .json")
7             val json = inputStream.bufferedReader().use {
8                 it.readText()
9             }
10
11             val gson = Gson()
12             val questions: List<Question> =
13                 gson.fromJson(json, object : TypeToken<List<
14                     ↪ Question>>(){}).type)
15
16             questions
17         } catch (e: Exception) {
18             emptyList()
19         }
20     }
21 }
```

Listing 5: QuestionLoader class structure

4.1.1 Error Handling

The loader implements defensive error handling:

- Wraps asset loading in try-catch
- Returns empty list on any exception (IOException, JsonSyntaxException)
- Prevents application crashes from malformed JSON

4.2 Question Repository

The QuestionRepository provides a clean abstraction over data sources and implements filtering logic:

```
1 class QuestionRepository(private val loader: QuestionLoader) {
2
3     fun getAllQuestions(): List<Question> {
4         return loader.loadQuestions()
5     }
6
7     fun getQuestionsByCategoryAndDifficulty(
8         category: Category,
9         difficulty: Difficulty
10    ): List<Question> {
11        return loader.loadQuestions()
12            .filter { it.category == category && it.difficulty ==
13                ↪ difficulty }
14    }
```

```
13     }
14 }
```

Listing 6: QuestionRepository interface and implementation

4.2.1 Repository Benefits

- **Abstraction:** Hides data source implementation details
- **Testability:** Easy to mock for unit tests
- **Flexibility:** Can switch between JSON, database, or API without UI changes
- **Filtering:** Provides domain-specific query methods

4.3 Image Resource Mapping

Images are referenced by name string in the JSON and resolved at runtime using Android's resource resolution:

```
1 fun getImageResId(context: Context, imageName: String): Int {
2     return context.resources.getIdentifier(
3         imageName,
4         "drawable",
5         context.packageName
6     )
7 }
```

Listing 7: Image resource resolution helper

This approach enables:

- JSON-driven image assignments
- Runtime validation of drawable existence
- Safe fallback to default resource if not found

5 Data Format & JSON Schema

5.1 Asset File Location

Questions are stored in: `app/src/main/assets/roman_questions.json`

5.2 JSON Schema

```
1 [
2   {
3     "id": 1,
4     "image": "eljem",
5     "question": "What is the Roman name for Tunisia?",
6     "options": ["Africa", "Numidia", "Mauretania", "Egypt"],
```

```

7      "correctAnswer": 0,
8      "category": "ROMAN",
9      "difficulty": "EASY"
10   },
11   {
12       "id": 2,
13       "image": "dougga",
14       "question": "Which Roman emperor visited Dougga?",
15       "options": ["Augustus", "Caracalla", "Trajan", "Hadrian"],
16       "correctAnswer": 1,
17       "category": "ROMAN",
18       "difficulty": "HARD"
19   }
20 ]

```

Listing 8: Sample question JSON structure

5.2.1 JSON Field Mapping

Fields in the JSON directly correspond to `Question` data class properties:

- String enum values ("ROMAN", "EASY") are automatically deserialized by Gson
- `correctAnswer` is a 0-based index into the `options` array
- `image` should reference a drawable resource (without file extension)

6 Technology Stack & Dependencies

6.1 Build Configuration

Table 2: Project Dependencies

Component	Version
Kotlin	2.0.21
Android Gradle Plugin (AGP)	9.0.1
Compose BOM	2024.11.01
Jetpack Compose Material3	Latest via BOM
Navigation Compose	Latest via catalog
ViewModel (lifecycle)	Latest via catalog
Gson	2.8.9+
Java Target	11
Min SDK	24
Target SDK	34

6.2 Key Libraries

- **Jetpack Compose:** Modern declarative UI framework

- **Navigation Compose:** Type-safe navigation routing
- **Gson:** JSON serialization/deserialization
- **ViewModel:** Lifecycle-aware state management
- **StateFlow:** Reactive state container

7 Usage Examples

7.1 Loading Questions in ViewModel

```
1 class QuizViewModel(  
2     private val repository: QuestionRepository  
3 ) : ViewModel() {  
4  
5     private val _questions = MutableStateFlow<List<Question>>(  
6         ↪ emptyList()  
7     )  
8     val questions: StateFlow<List<Question>> = _questions.  
9         ↪ asStateFlow()  
10  
11     fun loadQuestions(category: Category, difficulty: Difficulty)  
12         ↪ {  
13         viewModelScope.launch {  
14             _questions.value = repository  
15                 .getQuestionsByCategoryAndDifficulty(category,  
16                     ↪ difficulty)  
17         }  
18     }  
19 }
```

Listing 9: Using repository in ViewModel (future implementation)

7.2 Dependency Injection Pattern

```
1 val loader = QuestionLoader(context)  
2 val repository = QuestionRepository(loader)  
3 val viewModel = QuizViewModel(repository)  
4  
5 // Or with dependency injection framework (future):  
6 // val repository = hilt_inject<QuestionRepository>()
```

Listing 10: Instantiation in MainActivity (example)

8 Testing Strategy

8.1 Unit Testing Approach

- **Loader Tests:** Verify JSON parsing with mock assets

- **Repository Tests:** Mock loader and verify filtering logic
- **Model Tests:** Validate enum behavior and data class properties

8.2 Integration Testing

- End-to-end loading: Asset $\rightarrow$ Loader $\rightarrow$ Repository $\rightarrow$ ViewModel
- Real JSON asset validation in instrumented tests

9 Future Enhancements & Roadmap

9.1 Short Term

1. Complete ViewModel wiring to repository
2. Implement navigation flow between screens
3. Add unit and integration tests
4. Wire UI buttons to navigation and ViewModel

9.2 Medium Term

1. Migrate to Room database for local persistence
2. Implement user progress tracking
3. Add analytics and telemetry
4. Implement dependency injection with Hilt

9.3 Long Term

1. Remote API integration for dynamic content
2. Multiplayer quiz mode
3. Leaderboard system
4. Localization (AR, FR, EN)
5. Offline-first architecture with sync

10 Conclusion

TuniQuest provides a solid, production-ready foundation for an educational quiz application. The clean architecture, clear separation of concerns, and modern technology stack enable:

- **Maintainability:** Easy to understand and modify

- **Testability:** Well-designed components for unit testing
- **Scalability:** Ready for additional features and data sources
- **Developer Experience:** Modern Kotlin and Compose reduce boilerplate

The data layer implementation using JSON assets and the repository pattern establishes best practices that will serve the project as it evolves.

References

- [1] Google. *Jetpack Compose Documentation*. Available at: <https://developer.android.com/jetpack/compose>
- [2] Microsoft Patterns & Practices. *MVVM Architecture Pattern*. Available at: <https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm>
- [3] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [4] Google. *Gson: Java Serialization Library*. Available at: <https://github.com/google/gson>
- [5] Google. *Android Architecture Components*. Available at: <https://developer.android.com/topic/libraries/architecture>

A Build & Deployment

A.1 Gradle Build Commands

```
1 # Compile debug APK
2 ./gradlew compileDebugKotlin
3
4 # Build debug APK
5 ./gradlew assembleDebug
6
7 # Run lint checks
8 ./gradlew lint
9
10 # Install and run on device
11 ./gradlew installDebug
12
13 # Full build with tests
14 ./gradlew build
```

Listing 11: Common build tasks

B Gradle Configuration

Key settings from app/build.gradle.kts:

```
1 android {
2     namespace = "com.example.mohamedbelgacem"
3     compileSdk = 34
4
5     defaultConfig {
6         applicationId = "com.example.mohamedbelgacem"
7         minSdk = 24
8         targetSdk = 34
9         versionCode = 1
10        versionName = "1.0.0"
11    }
12
13    compileOptions {
14        sourceCompatibility = JavaVersion.VERSION_11
15        targetCompatibility = JavaVersion.VERSION_11
16    }
17 }
```

Listing 12: Gradle module configuration

C Git Commit History

Recent commits:

- `data layer` – Implemented `QuestionLoader`, `QuestionRepository`, and question models
- `TuniQuest rebranding` – Updated app name, theme, and project structure
- `Initial Compose scaffold` – Created project foundation with MVVM and navigation